



# **Pontificia Universidad Católica Madre y Maestra PUCMM**

Asignatura:  
**Fundamentos de la programación**

Tema:  
**Reporte Common Permutation**

Estudiante:  
Joseph Manuel Ureña Durán **Id:**10164781  
Adrian Alexander Artilles Vargas **Id:** 10170038

Facilitadora:  
Damarys

Fecha:  
**2/06/2026**

## 1. Análisis (Pensamiento Computacional)

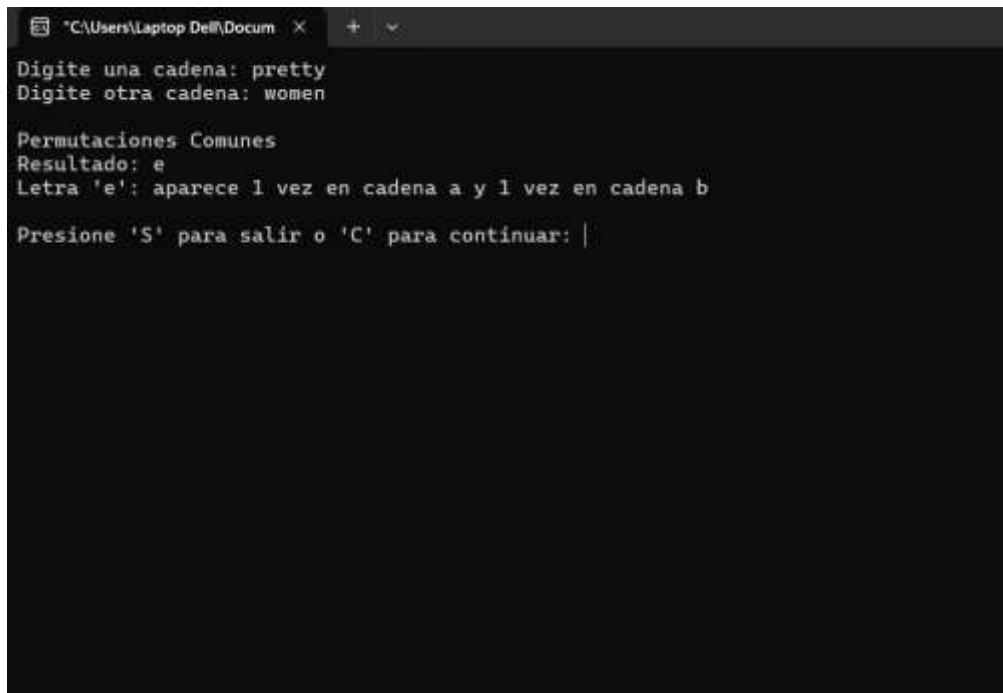
Para resolver el problema de la permutación común, aplicamos los principios del pensamiento computacional enfocándonos fuertemente en la manipulación manual de la memoria, tal como requiere el lenguaje C.

- **Abstracción:** Visualizamos las cadenas de caracteres no como texto simple, sino como bloques lineales de memoria RAM de 1 byte por posición, con un límite físico de 1000 elementos. Para lograr abstraer la información útil del "ruido", utilizamos el código ASCII. Esto nos permitió ignorar por completo los espacios en blanco y caracteres especiales mediante validaciones lógicas algebraicas (ej.  $*(a+i) < 'a'$ ). Además, comprendimos que el carácter nulo (`'\0'`) actúa como el portal indispensable; es la frontera que le indica al procesador dónde termina nuestra información válida y dónde comienza la memoria "basura", permitiéndonos aislar también los saltos de línea (`'\n'`) capturados por el teclado al usar `fgets`.
- **Descomposición:** En lugar de abordar el problema en un solo bloque complejo, dividimos la lógica en un flujo secuencial estricto mediante el uso de funciones separadas:
  1. *Captura y Estandarización:* Leemos ambas cadenas y utilizamos una función (`min`) para nivelar todo a minúsculas sumando 32 a las direcciones de memoria que contienen letras mayúsculas.
  2. *Intersección Bruta:* Iteramos sobre la primera palabra y verificamos si cada letra existe en la segunda (confirmar). Si hay coincidencia, almacenamos el carácter en una nueva cadena temporal en memoria.
  3. *Limpieza de Duplicados:* Para no repetir letras innecesariamente en nuestro conteo, la función repetida recorre la cadena resultante y, al encontrar caracteres idénticos, desplaza la memoria manualmente un espacio hacia la izquierda para sobrescribir y eliminar la repetición.
  4. *Ordenamiento:* Mediante la función alfabeto, comparamos los valores ASCII de los caracteres resultantes y los intercambiamos usando una variable temporal para garantizar que la subsecuencia quede en estricto orden alfabético.
  5. *Conteo e Interfaz:* Finalmente, la función permutaciones cuenta la frecuencia final en las cadenas base y el bloque main orquesta que el usuario decida si presionar 'C' para continuar o 'S' para salir.
- **Reconocimiento de Patrones:** Al analizar la lógica de búsqueda, notamos una similitud inherente: el proceso de avanzar byte por byte sobre la primera palabra para evaluar un carácter es computacionalmente idéntico al proceso requerido para la segunda palabra. Este patrón nos permitió reutilizar la misma lógica de indirección y avance de punteros (`puntero++`) de forma simétrica para ambas variables, garantizando que el tiempo de ejecución se mantuviera eficiente y predecible.

## 2. Ejemplos de Corrida

A continuación, demostramos que el programa tiene un comportamiento robusto en la terminal ante distintos escenarios:

### A. Caso Exitoso Estándar

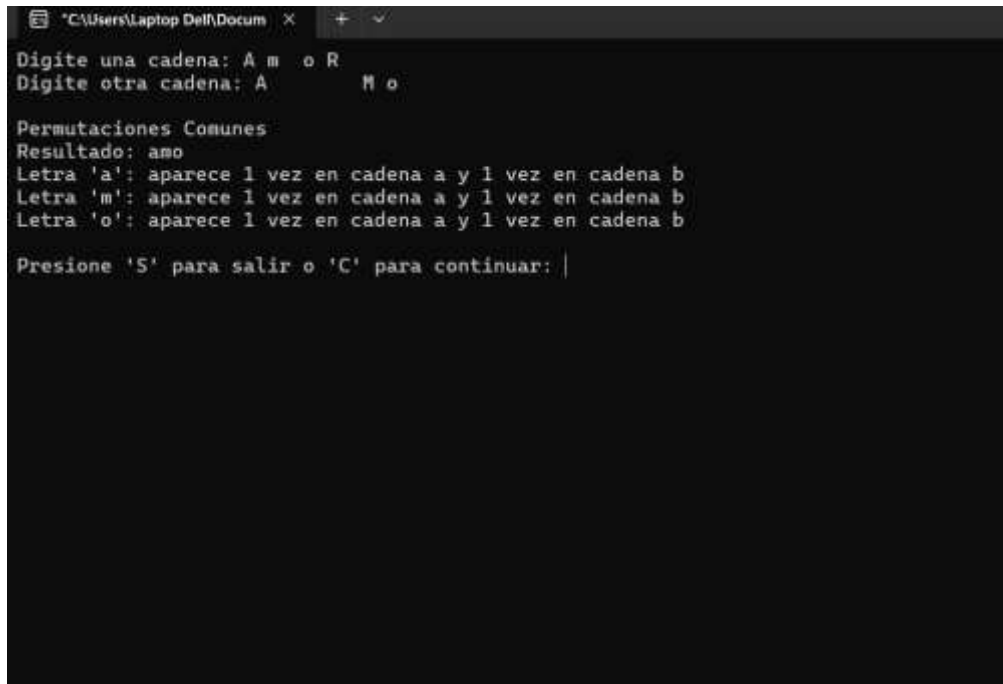


```
"C:\Users\Laptop Dell\Docum  X  +  v
Digite una cadena: pretty
Digite otra cadena: women

Permutaciones Comunes
Resultado: e
Letra 'e': aparece 1 vez en cadena a y 1 vez en cadena b

Presione 'S' para salir o 'C' para continuar: |
```

### B. Caso con Mezcla de Formatos (Limpieza)



```
"C:\Users\Laptop Dell\Docum  X  +  v
Digite una cadena: A m o R
Digite otra cadena: A M o

Permutaciones Comunes
Resultado: amo
Letra 'a': aparece 1 vez en cadena a y 1 vez en cadena b
Letra 'm': aparece 1 vez en cadena a y 1 vez en cadena b
Letra 'o': aparece 1 vez en cadena a y 1 vez en cadena b

Presione 'S' para salir o 'C' para continuar: |
```

### C. Caso de Control de Interfaz y Robustez

```
"C:\Users\Laptop Dell\Docum  X  +  v
Digite una cadena: pretty
Digite otra cadena: woman

Permutaciones Comunes
Resultado:

Presione 'S' para salir o 'C' para continuar: f
Opcion invalida.
Presione 'S' para salir o 'C' para continuar: v
Opcion invalida.
Presione 'S' para salir o 'C' para continuar: |
```

```
"C:\Users\Laptop Dell\Docum  X  +  v
Digite una cadena: pretty
Digite otra cadena: woman

Permutaciones Comunes
Resultado:

Presione 'S' para salir o 'C' para continuar: f
Opcion invalida.
Presione 'S' para salir o 'C' para continuar: v
Opcion invalida.
Presione 'S' para salir o 'C' para continuar: s
Saliendo del programa...

Process returned 0 (0x0)   execution time : 36.304 s
Press any key to continue.
|
```

### 3. Conclusiones Tecnológicas

Trabajar la aritmética de punteros nos permitió entender la diferencia abismal entre programar a alto nivel y controlar el hardware a bajo nivel. Utilizar corchetes (`cadena[i]`) es esencialmente "azúcar sintáctica" que le pide al compilador calcular la dirección de memoria por nosotros. Sin embargo, utilizar indirección (`*puntero`) y avance secuencial (`puntero++`) nos otorga un control real y directo sobre la computadora.

Este enfoque directo nos obligó a ser sumamente cuidadosos. Al interactuar físicamente con la RAM, comprendimos que omitir un carácter nulo (`'\0'`) o manipular bloques de memoria que se solapan (overlap) produce corrupciones de memoria (comportamiento indefinido). En conclusión, dominar los punteros nos hace programadores más conscientes de la eficiencia algorítmica y de la seguridad de nuestro software frente a posibles vulnerabilidades.

### 4. Transparencia de Autoría (Uso de IA)

Declaramos de forma honesta que durante el desarrollo de este código utilizamos herramientas de Inteligencia Artificial (Gemini) estrictamente como apoyo de depuración (debugging) y no como generación automática de lógica matemática, la cual es de autoría propia.

Nuestra interacción con la IA se limitó a comprender las mejores prácticas en C frente a errores de ejecución. Específicamente, utilizamos los siguientes comandos/prompts:

- **Prompt 1:** "Revisa bugs y problemas de lógica en este código de C".
- **Conceptos aclarados:** La herramienta nos orientó sobre por qué utilizar funciones de la librería `<string.h>` como `strcpy` en bloques de memoria que se solapan (como desplazar un arreglo sobre sí mismo para borrar un duplicado) puede causar un comportamiento indefinido en C. Con este conocimiento, sustituimos esa función por un algoritmo de desplazamiento manual seguro utilizando punteros. Además, la IA nos reforzó la obligatoriedad teórica de insertar manualmente el terminador nulo (`'\0'`) al construir cadenas carácter por carácter para evitar imprimir basura contenida previamente en la RAM.